

FlowLight

AI 에이전트 기반 실시간 교통신호 최적화 시스템

고정형 신호 체계의 한계를 LLM Agent 기반 분석·계획·평가 구조로 개선

문제

도심 정체와
고정 신호의 한계

과정

LLM Agent가
분석·계획·평가

결과

AI 적용 전후
교통 흐름 비교

목차

문제 → 과정 → 결과 → 회고 흐름으로 구성

01

프로젝트 개요 및 문제 정의

도심 정체와 고정형 신호 체계의 한계

02

관련 방식과 한계

고정 신호·센서 기반·강화학습 방식 비교

03

데이터 준비

CityFlow 기반 교통 상태 생성 및 JSON 변환

04

핵심 솔루션

FlowLight 작동 원리와 차별성

05

기술 아키텍처

전체 데이터 흐름과 시스템 구성

06

핵심 구현

LLM Agent·Prompt·Guardrail·FastAPI/SSE

07

실험 및 라이브 데모

AI 적용 전후 영상 비교와 결과 분석

08

배운 점·한계·향후 계획

교육 과정에서의 학습과 개선 방향

발표 핵심 메시지

현직자 피드백을 반영한 문제·과정·결과 중심 구조

문제

고정형 신호는 차량 흐름 변화에
실시간 대응하기 어려움

출퇴근·혼잡 상황에서 특정 방향 대기열이 누적되고,
빈 도로에 초록불이 켜지는 비효율이 발생

과정

AI가 교통 상태를 이해하고 신호 계획을
생성하도록 설계



결과

AI 적용 전후를 비교하여 교통 흐름 개선
가능성을 확인

동일 조건에서 고정 신호와 AI 적응형 신호를 영상과
지표로 비교하여 개선 효과를 검증

발표의 결론: “AI를 붙였다”가 아니라, AI가 분석·계획·평가를 수행해 신호 제어 의사결정에 참여했다는 점을 보여줌.

프로젝트 개요 및 문제 정의

01 문제 정의

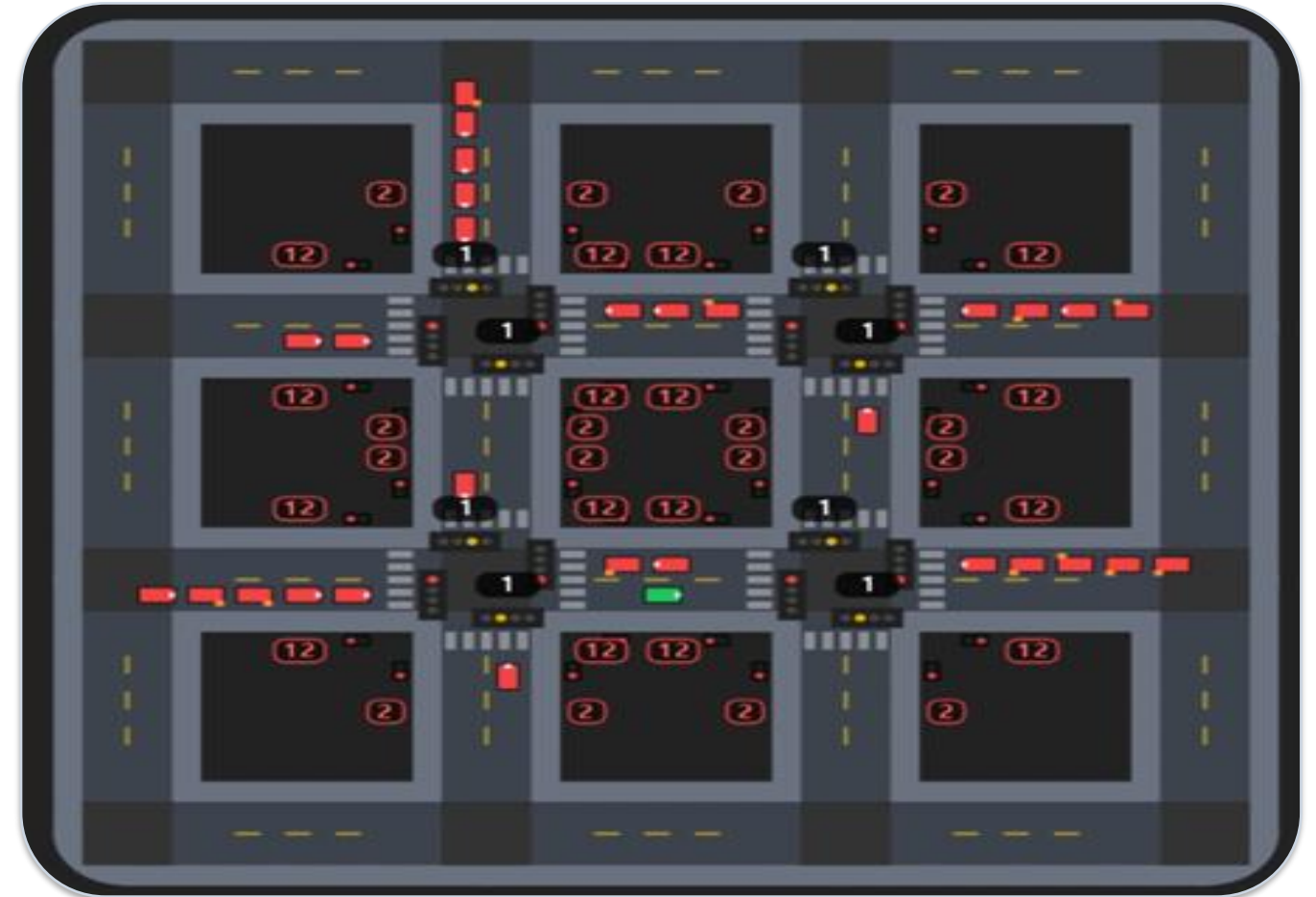
Introduction | 왜 이 문제를 해결하려 했는가?

도심 교통 정체의 심화

- 차량 증가와 복잡한 도로망으로 교차로 혼잡이 지속적으로 발생
- 대기 시간 증가, 공회전, 연료 낭비와 같은 사회적 비용 발생
- 특정 방향에 차량이 몰릴 경우 정체가 빠르게 확산

고정형 신호 체계의 한계

- 대부분의 신호는 도로 상황과 무관하게 정해진 주기로 운영
- 차량이 없는 방향에도 신호 시간이 배정되는 비효율 발생
- 출퇴근·혼잡·돌발 상황에 즉각 대응하기 어려움



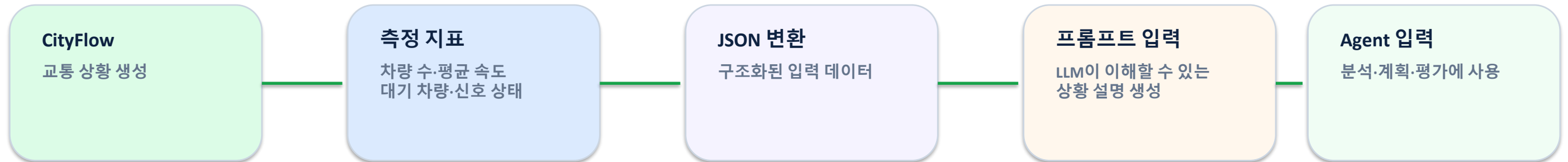
프로젝트 목표: LLM Agent가 실시간 교통 상황을 분석하고, 최적의 신호 시간을 자동 생성하여 교차로 정체를 완화

기존 방식과 한계

Related Work | 기존 접근 방식과 FlowLight의 차별점

기존 방식	장점	한계	FlowLight 관점
고정형 신호	구현이 단순하고 안정적	실시간 교통량 변화 대응이 어려움	기본 비교 대상(Baseline)으로 사용
센서 기반 제어	현장 데이터 반영 가능	설치·유지 비용과 인프라 의존도 큼	시뮬레이션 데이터로 AI 판단 구조 검증
강화학습 기반 제어	장기 최적화 가능	학습 환경·시간·데이터 요구량이 큼	향후 고도화 방향으로 제안
LLM Agent 방식	상황 해석과 의사결정 흐름 설명 가능	응답 품질·안전 검증 필요	분석·계획·평가·Guardrail 구조로 보완

핵심 차별점: 단순 규칙이 아니라 LLM Agent가 교통 상황을 분석하고, 신호 계획을 생성·평가한 뒤 안전 검증을 거쳐 적용



입력 데이터 예시(JSON)

```
{
  "total_cars": 29,
  "stopped_cars": 17,
  "pedestrians": 0,
  "congestion": 0.80,
  "cycle_sec": 20
}
```

🚗 차량 수 : 29대
🚦 정지 차량 : 17대
🚶 보행자 : 0명
📶 혼잡도 : 0.80 (매우 높음)
🕒 신호 주기 : 20초

데이터 처리 기준

- 복잡한 시뮬레이션 객체 전체가 아니라 핵심 교통 지표만 추출
- LLM이 안정적으로 해석할 수 있도록 JSON 형태로 구조화
- 실험 조건 비교를 위해 동일 시나리오·동일 Seed 조건을 활용
- AI 적용 전·후 결과를 평균 속도, 대기 차량 수, 통행량 등으로 비교

핵심 솔루션: FlowLight

Method Overview | AI가 교통 상황을 보고 신호 계획을 생성하는 구조

04 솔루션



핵심 아이디어

- AI가 현재 교통 상황을 요약하고 혼잡 방향을 판단
- 대기 차량이 많은 방향의 녹색 시간을 조정
- 생성된 계획을 평가하여 기준 미달 시 재생성

차별성

- LLM Agent 기반 분석·계획·평가 구조
- Prompt Engineering으로 JSON 신호 계획 생성
- Guardrail로 비정상 값과 안전성 문제를 필터링

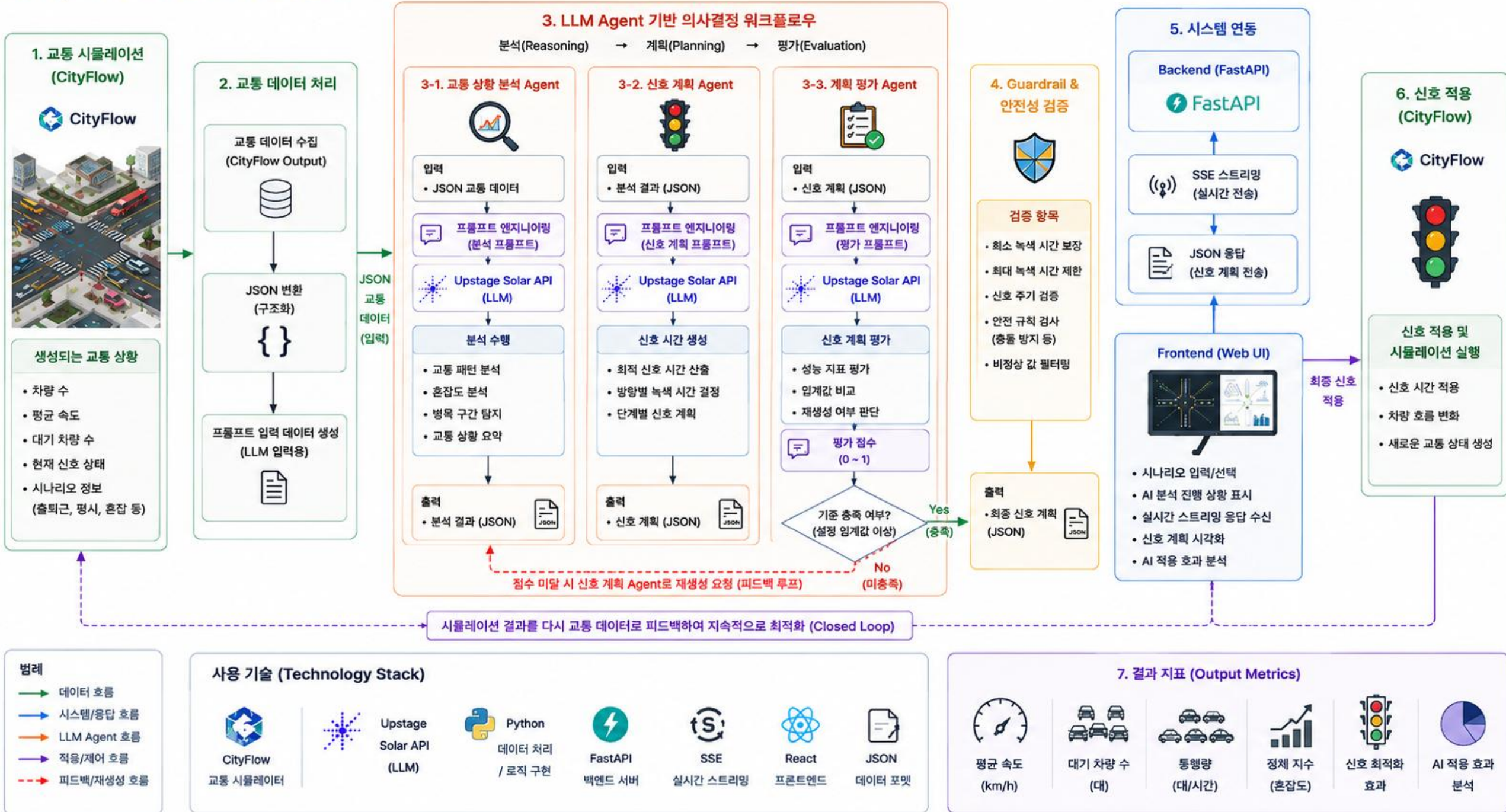
구현 결과

- FastAPI 서버와 Frontend 연동
- SSE로 AI 분석 과정을 실시간 스트리밍
- AI 적용 전·후 영상을 비교하는 데모 구성

FlowLight 시스템 아키텍처

LLM Agent 기반 교통 신호 최적화 전체 흐름도

→ 데이터 흐름 → 시스템/응답 흐름 → LLM Agent 흐름 → 적용/제어 흐름 - - - 피드백/재생성 흐름



기술 아키텍처 상세 파이프라인

05 아키텍처

데이터가 어떤 모듈을 거쳐 결과가 되는지 단계별로 정리



1. 경량화된 상태 전달

- 초당 반복되는 시뮬레이션 데이터를 그대로 보내지 않음
- 차량 수, 평균 속도, 대기 차량 등 핵심 메트릭만 추출
- LLM 입력에 필요한 정보만 JSON으로 직렬화

2. 비동기 응답 처리

- AI 응답 대기 중에도 화면이 멈추지 않도록 구성
- FastAPI와 SSE를 이용해 진행 상태를 순차적으로 전달
- 사용자는 분석 과정과 최종 신호 계획을 실시간 확인

3. 규격화된 결과 적용

- Solar API가 생성한 신호 계획을 JSON으로 받음
- 평가 점수와 Guardrail을 통해 적용 가능 여부 검증
- 검증된 신호 계획만 시뮬레이션에 반영

핵심 구현 ① LLM Agent Workflow

06 핵심 구현

교통 상황 분석 → 신호 계획 → 계획 평가의 의사결정 구조

교통 상황 분석 Agent

입력: JSON 교통 데이터

- 교통 패턴 분석
- 혼잡도 분석
- 병목 구간 파악
- 교통 상황 요약

신호 계획 Agent

입력: 분석 결과(JSON)

- 신호 시간 생성
- 방향별 녹색 시간 결정
- 단계별 신호 계획 생성
- JSON 형식 출력

계획 평가 Agent

입력: 신호 계획(JSON)

- 성능 지표 평가
- 임계값 비교
- 재생성 여부 판단
- 피드백 생성

평가 점수 미달 시 신호 계획 Agent로 재생성 요청

핵심 구현 ② 프롬프트 설계

06 핵심 구현

Signal Planning Agent가 입력값 기반으로 신호 시간을 생성하도록 설계

핵심은 “LLM에게 역할·입력·판단 규칙·출력 형식”을 명확히 주어, 매번 현재 교통 상태에 맞는 신호 계획을 생성하게 하는 것

프롬프트 구성 예시

Role FlowLight 신호 계획 Agent

Input 차량 수 / 정지 차량 수
보행자 수 / 혼잡도 / 신호 주기

Rule 보행자 없음 → 차량 우선
혼잡도 높음 → 차량 녹색 증가
보행자 증가 → 보행자 녹색 증가

제약 전체 주기 초과 금지
최소 신호 시간 준수
JSON 형식으로만 출력

Output 신호 계획(JSON)

입력 데이터 예시

 차량 수
29대

 정지 차량
17대

 보행자
0명

 혼잡도
0.80

```
{  
  "total_cars": 29,  
  "stopped_cars": 17,  
  "pedestrians": 0,  
  "congestion": 0.80  
}
```

출력 형식 및 결과

AI 신호 제어안

```
{  
  "priority": "VEHICLE",  
  "north_south": 12,  
  "east_west": 8,  
  "pedestrian": 0  
}
```

판단 근거

보행자가 없고 혼잡도가 높아
차량 우선 제어로 신호 시간을 배분

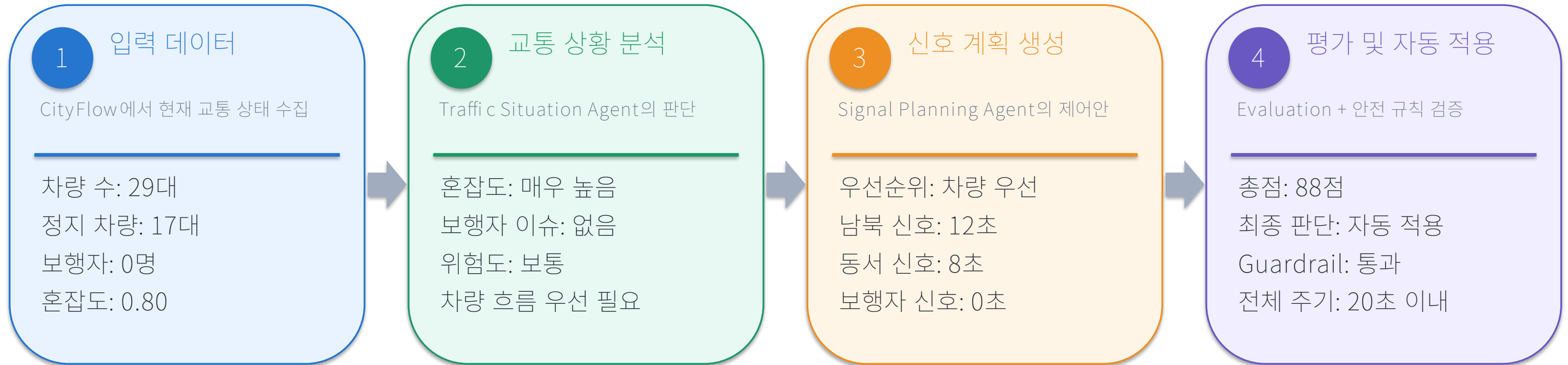
프롬프트는 단순 질문이 아니라, AI가 “무엇을 보고, 어떤 규칙으로, 어떤 형식으로” 판단할지 정의한 설계 문서

AI 의사결정 과정

06 핵심 구현

입력 데이터 → 상황 분석 → 신호 계획 → 평가 및 자동 적용

FlowLight는 교통 데이터를 분석하고, 신호 계획을 생성한 뒤 평가와 안전 규칙 검증을 거쳐 최종 적용 여부를 결정



보행자가 없고 혼잡도가 높기 때문에 차량 우선 제어를 선택하고, 전체 주기 20초 이내에서 남북 12초 / 동서 8초로 신호를 조정

핵심 구현 ③ 평가와 Guardrail

06 핵심 구현

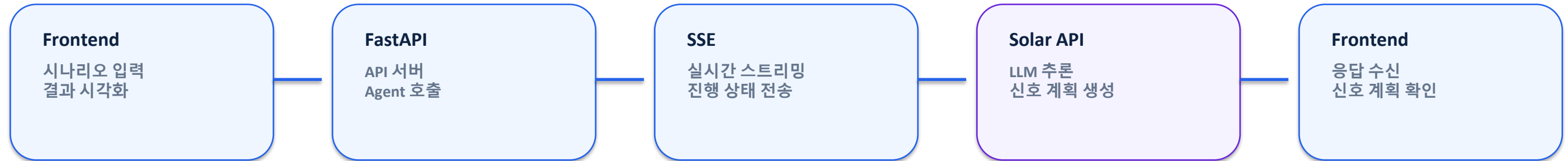
LLM 결과를 그대로 적용하지 않고 평가·검증 후 적용



핵심 구현 ④ 시스템 운영화

06 핵심 구현

FastAPI와 SSE를 활용한 실시간 AI 응답 연동



구현 목적

- AI 응답 시간이 발생해도 화면이 멈추지 않도록 비동기 구조 적용
- 사용자에게 분석 시작, 계획 생성, 평가 결과를 단계별로 전달
- 로컬 서버 환경에서 데모가 안정적으로 실행되도록 예외 처리 보완

분석 리포트

AI 에이전트 분석을 시작합니다...

[진행] 시뮬레이션 데이터 수신 완료

[Agent 1] Traffic Situation Agent
[진행] 교통 상황 분석 시작

[Agent 2] Signal Planning Agent
[진행] 신호 계획 생성 시작

[Guardrail] Safety Rule Check
[진행] 최소 신호 시간 Guardrail 적용 완료

[Agent 3] Plan Evaluation Agent
[진행] 계획 평가 시작

=====

FlowLight AI 분석 결과

=====

[교통 상황]

- 차량 수: 23대
- 정지 차량: 18대
- 보행자: 0명
- 혼잡도: 매우 높음 (1.00)

[AI 신호 제어안]

- 우선순위: 차량 우선
- 남북 신호: 12초
- 동서 신호: 8초
- 보행자 요청: 없음
- 전체 주기: 20초

[AI 판단 근거]

- 보행자가 없어 차량 흐름을 우선하도록 신호를 조정했습니다.
- 최종 신호시간 : 남북 12초 / 동서 8초 / 보행자 0초
- Guardrail 검증 : 전체 주기 20초 이내로 안전하게 조정했습니다.

[평가 결과]

- 총점: 88점

Experiments | AI 적용 전후를 동일 조건에서 비교

지표	고정 신호	AI 적용	변화 방향
대기 차량 수	18대	13대	감소 
통행량	107대/분	115대/분	증가 
정체 지수	1.00	0.76	감소 

AI 적용 결과

적용 여부: 자동 적용 완료
적용 교차로: 4개
차량 제어: 남북 12초 / 동서 8초
보행자 제어: 보행자 요청 없음 → 빨간불 고정
Guardrail: 최소 신호 시간 및 주기 초과 검증 완료

AI 적용 효과

처리량 : 107대/분 → 115대/분
정지 차량 : 18대 → 13대
정체 상태 : 1.00 → 0.76
판단 : 처리량 증가 및 정체 완화 확인

발표에서 강조할 문장

- 단순히 AI를 연결한 것이 아니라, 동일 조건에서 고정 신호와 AI 신호를 비교했다.
- AI가 교통 상황을 분석하여 신호 시간을 조정했고, 그 결과 교통 흐름 개선 가능성을 확인했다.
- 실제 수치는 최종 시뮬레이션 결과로 교체한다.

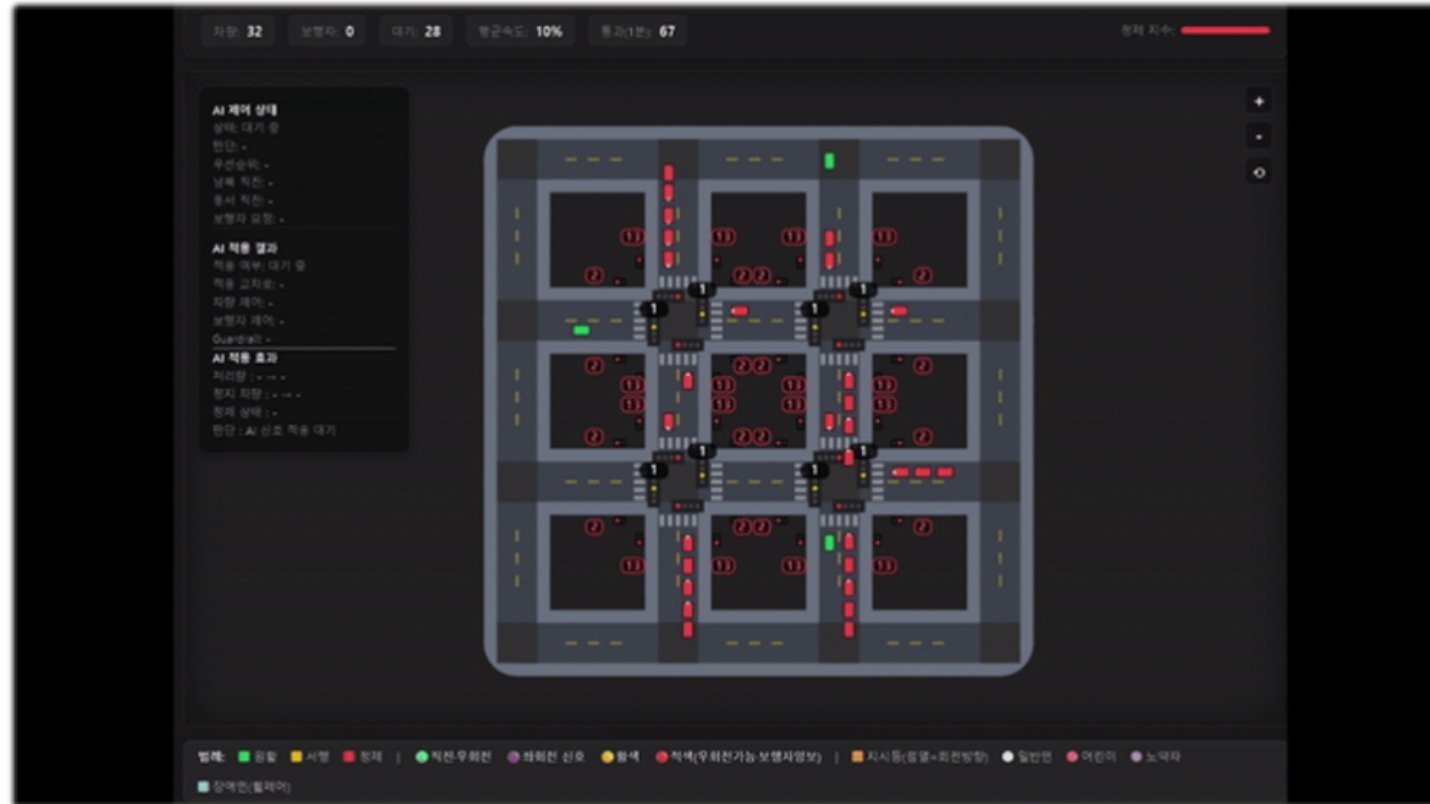
라이브 데모 및 A/B 테스트 검증

07 라이브 데모

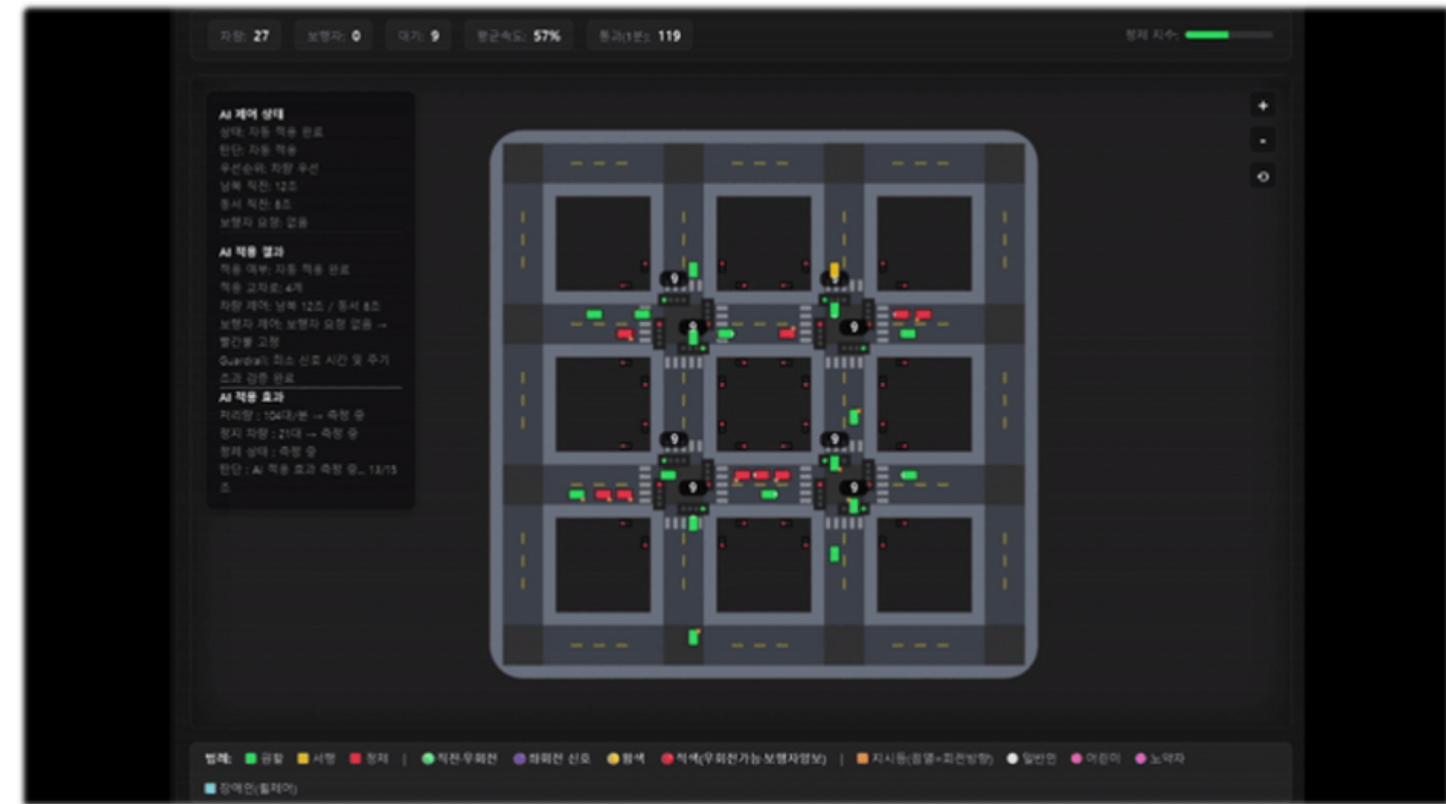
AI 적용 전 영상과 AI 적용 후 영상을 같은 조건에서 비교

고정 신호

AI 판단 적용 신호



VS



동일 조건

차량 수·시드·시나리오 고정

비교 포인트

대기 차량·차량 흐름·정체 상태

핵심 메시지

AI가 신호 제어 의사결정에 참여해 개선 효과 확인

교통량 (대/초) : 3 / 보행자 (명/초) : 0 / 최대 차량 (대) : 40 / 최고속도 (px/s):60 / 격자 크기 (NxN): 4

AI 시스템 설계

- LLM을 단순 호출하는 것과 Agent 구조로 설계하는 것의 차이를 이해
- 분석·계획·평가를 분리하면 AI 의사결정 흐름을 설명하기 쉬워짐
- Prompt와 JSON 형식 설계가 결과 품질에 직접적인 영향을 준다는 점을 학습

서비스 구현

- FastAPI를 이용해 AI 추론 서버를 구성
- SSE를 통해 분석 진행 상황을 실시간으로 전달
- Frontend와 Backend를 연결하면서 API 설계와 예외 처리의 중요성 확인

검증과 운영 관점

- AI가 생성한 값을 그대로 적용하면 위험할 수 있음을 확인
- 평가 점수와 Guardrail을 통해 안전한 결과만 적용하도록 보완
- 실험 결과를 수치와 영상으로 함께 보여주는 것이 설득력 있다는 점을 학습

프로젝트 전에는 “AI를 연결하면 된다”고 생각했지만,
실제 구현 과정에서 AI 결과를 평가·검증하고 서비스 흐름에 안전하게 연결하는 과정이 더 중요하다는 것을 배웠다.

Limitation & Future Work | 무엇이 아쉬웠고, 다음에는 어떻게 개선할 것인가?

아쉬웠던 점	구현하지 못한 이유	개선 방향
실제 교통 데이터 연동	공공 API·실시간 센서 데이터 확보 및 정제 시간이 부족	공공 교통 API·CCTV/센서 데이터 연동
회전교차로 시나리오	교통 규칙과 우선순위 모델링이 일반 교차로보다 복잡	회전교차로 전용 차량 흐름 규칙 추가
다중 교차로 확장	교차로 간 영향 관계와 동기화 로직 구현 시간이 부족	인접 교차로 간 데이터 공유 및 녹색파 구현
강화학습 적용	학습 환경 구축과 충분한 반복 실험 시간이 부족	LLM Agent와 강화학습을 결합한 하이브리드 최적화

향후 로드맵: 실제 데이터 연동 → 회전교차로·다중 교차로 확장 → 강화학습 결합 → 스마트시티 교통 제어 시스템 고도화

Conclusion

FlowLight는 AI Agent가 교통 상황을 분석하고 신호 계획을 생성·평가하는 실시간 교통신호 최적화 시스템입니다.

문제

고정형 신호는 실시간 교통 변화에 대응하기 어려움

과정

CityFlow 데이터와 Upstage Solar API 기반 LLM Agent를 활용해 분석·계획·평가 구조를 구현

결과

AI 적용 전후 비교를 통해 교통 흐름 개선 가능성을 확인

회고

실제 데이터·회전교차로·다중 교차로·강화학습으로 확장 예정

Q & A

경청해 주셔서 감사합니다.